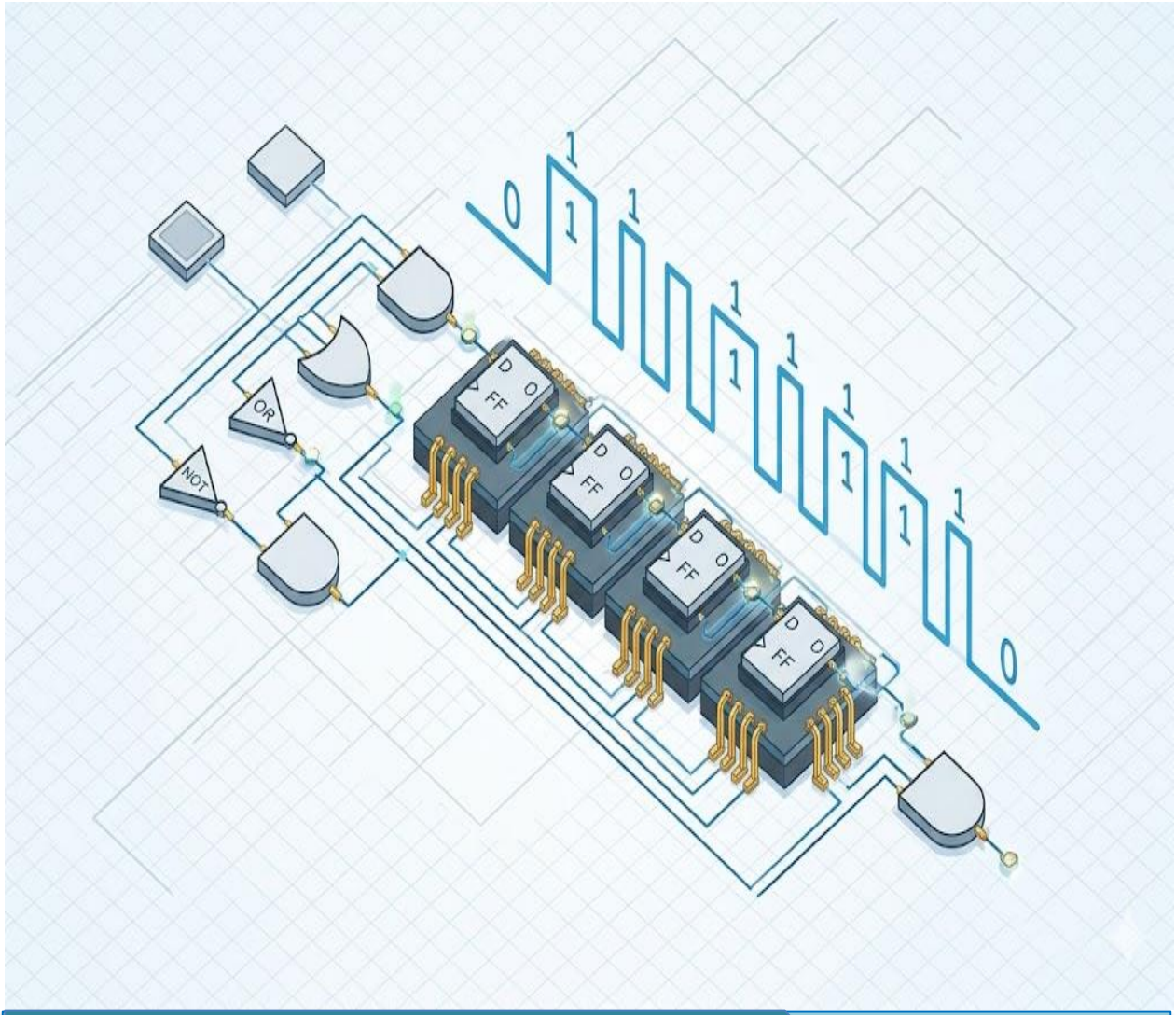


DLD OEL REPORT
TOPIC: SEQUENCE DETECTOR

- MALIHA SEEMI ANWAR (EE-24004)
- SANIA TARIQ (EE-24027)
- FAREENA KHAN (EE-24002)
- MAHA TARIQ (EE-24023)

OEL Objective:

Design and implement a sequence detector that meets the following specifications:
• The detector contains single input 'w' and output 'z'.
• All changes in the circuit occur on the positive edge of a clock signal.
• The output 'z' is equal to '1' if input 'w' receives sequence "01111110" in the immediately preceding clock cycles.

Introduction:

For this design, the **Mealy Machine** model was selected over the Moore Machine for the following reasons:

1. Reduced State Complexity

A Mealy machine typically requires **fewer states** than a Moore machine to detect the same sequence.

- In a Mealy machine, the output is associated with the **transition**.
- In a Moore machine, the output is associated with the **state** itself, often requiring an extra "success" state at the end.
- **Benefit:** By using Mealy, we kept the design to 8 states, allowing us to stay within a **3-bit flip-flop** (Q_2, Q_1, Q_0) architecture. A Moore machine might have pushed the design toward more complex state assignments or extra flip-flops.

2. Faster Response Time

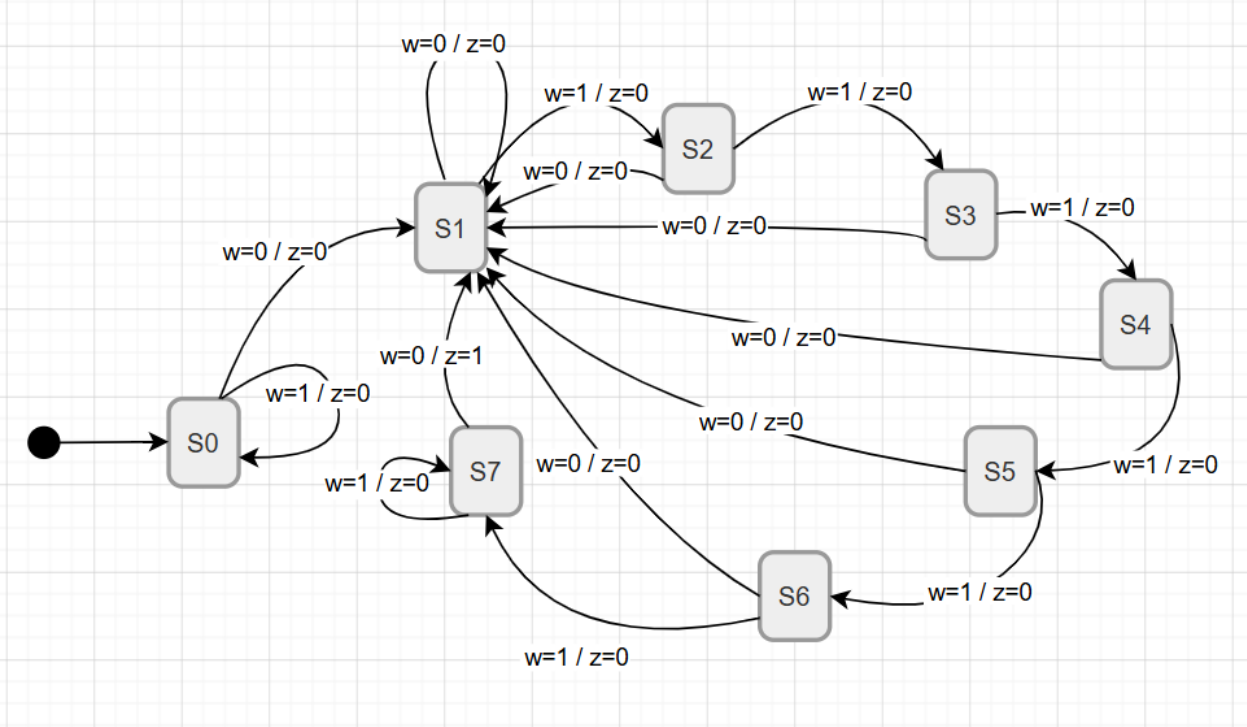
Because the output z in a Mealy machine is a function of both the present state and the current input (w), the circuit reacts **immediately** when the final bit of the sequence is detected.

- In a Moore machine, the output would only change on the **next clock edge** after entering the final state.
- **Benefit:** This provides a "real-time" detection feel, which is critical for high-speed sequence monitoring.

3. Simplified Combinational Logic

By utilizing the input ‘w’ directly in the output K-map, we can often produce a very simple output expression. This minimizes the total number of logic gates required on the breadboard compared to a Moore model, where the output logic would rely solely on state bits and potentially require more decoding gates.

State Diagram:



State Table:

Present State (Q2 Q1Q0)	Next State (w=0)	Next State (w=1)	Output z (w=0)	Output z (w=1)
S0 (000)	S1 (001)	S0 (000)	0	0
S1 (001)	S1 (001)	S2 (010)	0	0
S2 (010)	S1 (001)	S3 (011)	0	0
S3 (011)	S1 (001)	S4 (100)	0	0
S4 (100)	S1 (001)	S5 (101)	0	0
S5 (101)	S1 (001)	S6 (110)	0	0
S6 (110)	S1 (001)	S7 (111)	0	0
S7 (111)	S1 (001)	S7 (111)	1	0

Input/Output Combinational Circuit:

Present State (Q2Q1 Q0)	Input (w)	Next State / FF Inputs (D2 D1D0)	Output (z)
S0 (000)	0	001 (S1)	0
	1	000 (S0)	0
S1 (001)	0	001 (S1)	0
	1	010 (S2)	0
S2 (010)	0	001 (S1)	0
	1	011 (S3)	0
S3 (011)	0	001 (S1)	0
	1	100 (S4)	0
S4 (100)	0	001 (S1)	0
	1	101 (S5)	0
S5 (101)	0	001 (S1)	0
	1	110 (S6)	0
S6 (110)	0	001 (S1)	0
	1	111 (S7)	0
S7 (111)	0	001 (S1)	1
	1	111 (S7)	0

Input K-Maps:

D0:

Q1Q2\Q0w	00	01	11	10
00	1	0	1	1
01	1	0	1	1
11	1	0	1	1
10	1	0	1	1

Using POS:

$D_0 = Q_0 + w'$

D1:

Q1Q2\Q0w	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	0	1
10	0	0	0	1

Using SOP:

$$D_1 = wQ_0Q_1 + wQ_0Q_1'$$
$$D_1 = Q_0\,(w'Q_1 + wQ_1')$$
$$D_1 = Q_0\,(Q_1 \oplus w)$$

D2:

Q1Q2\Q0w	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	1	1	1
10	0	1	0	0

Using SOP:

$$D_2 = Q_0Q_2 + wQ_0'Q_1$$

Output K-Map:

z:

Q1Q2\Q0w	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	1	0

Using SOP:

$$z = wQ_0Q_1Q_2'$$

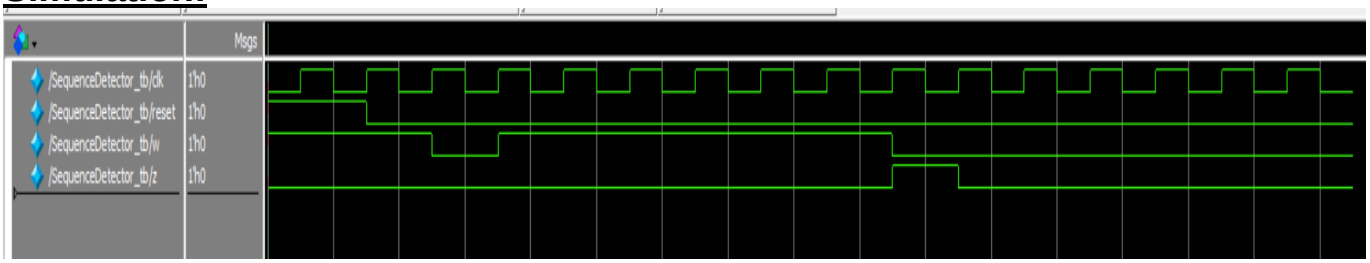
Verilog Code:

```
module SequenceDetector (
    input clk,
    input reset,
    input w,
    output reg z
);
    reg [2:0] state;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= 3'b000;
            z <= 1'b0;
        end else begin
            case (state)
                3'b000: state <= (w == 0) ? 3'b001 : 3'b000;
                3'b001: state <= (w == 1) ? 3'b010 : 3'b001;
                3'b010: state <= (w == 1) ? 3'b011 : 3'b001;
                3'b011: state <= (w == 1) ? 3'b100 : 3'b001;
                3'b100: state <= (w == 1) ? 3'b101 : 3'b001;
                3'b101: state <= (w == 1) ? 3'b110 : 3'b001;
                3'b110: state <= (w == 1) ? 3'b111 : 3'b001;
                3'b111: state <= (w == 0) ? 3'b001 : 3'b000;
                default: state <= 3'b000;
            endcase
            z <= (state == 3'b111 && w == 0);
        end
    end
endmodule
```

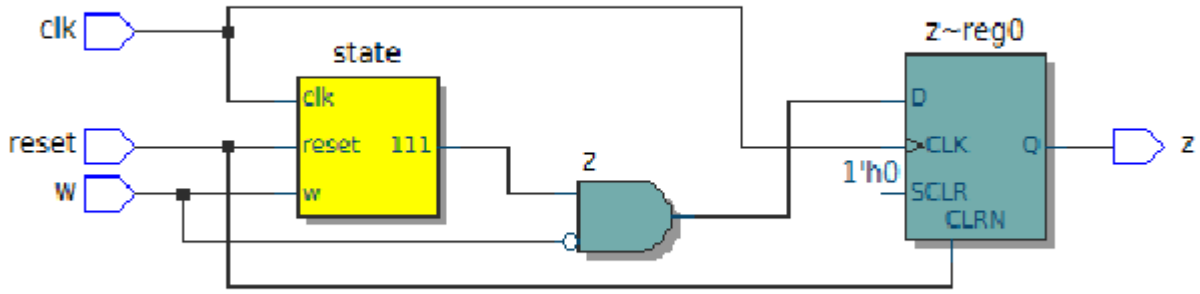
Testbench Code:

```
`timescale 1ns/1ps
module SequenceDetector_tb();
    reg clk, reset, w;
    wire z;
    SequenceDetector uut (.clk(clk), .reset(reset), .w(w), .z(z));
    always #5 clk = ~clk;
    initial begin
        clk = 0; reset = 1; w = 1;
        #15 reset = 0;
        #10 w = 0;
        #10 w = 1;
        #10 w = 1;
        #10 w = 1;
        #10 w = 1;
        #10 w = 1;
        #10 w = 1;
        #10 w = 0;
        #20 w = 0;
        #50 $stop;
    end
endmodule
```

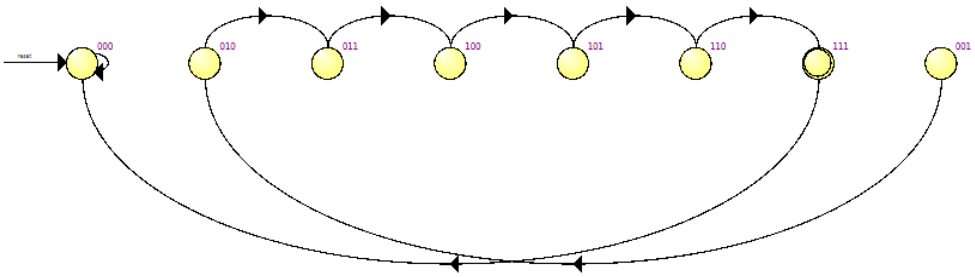
Simulation:



Schematics:



	Source State	Destination State	Condition
1	000	000	(w)
2	001	010	(w)
3	010	011	(w)
4	011	100	(w)
5	100	101	(w)
6	101	110	(w)
7	110	111	(w)
8	111	000	(w)



Conclusion:

The design and implementation of the **01111110** sequence detector were successfully completed using a **Mealy Machine** model. By choosing the Mealy architecture, the design was optimized to just **8 states**, allowing for a compact **3-bit flip-flop** implementation. This approach not only reduced the hardware complexity and the number of logic gates required but also ensured **real-time output detection** upon receiving the final bit. The use of K-maps further simplified the combinational logic, resulting in an efficient, high-speed circuit that fully meets all the project specifications.